

Java 23 features

- [Java 23 New Features With Examples](#)
- [Primitive Types in Patterns, instanceof, and switch \(Preview\) – JEP 455](#)
 - [Primitive Types in instanceof:](#)
 - [Primitive Types in switch expressions:](#)
- [Markdown Documentation Comments – JEP 467](#)
 - [How to use Markdown Documentation Comments?](#)
 - [Example \(Traditional vs. Markdown\)](#)
- [Module Import Declarations \(Preview\) – JEP 476](#)
- [Implicitly Declared Classes and Instance Main Methods \(Third Preview\) – JEP 477](#)
- [Flexible Constructor Bodies \(Second Preview\) – JEP 482](#)
- [Class-File API \(Second Preview\) – JEP 466](#)
 - [Example:](#)
- [Stream Gatherers \(Second Preview\) – JEP 473](#)
 - [Example:](#)
- [Structured Concurrency \(Third Preview\) – JEP 480](#)
 - [Example:](#)
- [Scoped Values \(Third Preview\) – JEP 481](#)
 - [Example:](#)
- [Vector API \(Eighth Incubator\) – JEP 469](#)
 - [Example:](#)
- [ZGC: Generational mode by default – JEP 474](#)
 - [Example:](#)

Java 23 New Features With Examples

Let's start exploring Java 23 New Features with examples one by one.

Primitive Types in Patterns, instanceof, and switch (Preview) – JEP 455

This is a preview language feature. Till Java 22, instanceof cannot be used with primitive data types at all.

We can explicitly enable preview features in the javac command using the VM options `--enable-preview` `--source 23`

[JDK Enhancement Proposal 455](#) introduces two major changes in Java 23:

- We can use all primitive types in switch expressions and statements, including long, float, double, and boolean.
- Pattern matching has been extended to support all primitive types for both instanceof and switch statements.

Primitive patterns have a different interpretation compared to object patterns because primitive types do not follow inheritance, unlike objects.

For example, consider the below code:

```
int i = ...

if (i instanceof byte b) {

    System.out.println("b = " + b);

}
```

The code should be read as follows: If the value of the variable i can also be stored in a byte variable, then assign this value to the byte variable b and print it.

This would be the case for value = 5, for example, but not for value = 1000, as byte can only store values from -128 to 127.

Just as with objects, for primitive types, we can also add further checks directly in the instanceof check using `&&`. The following code, for example, only prints positive byte values (i.e., 1 to 127):

```
int value = ...

if (value instanceof byte b && b > 0) {

    System.out.println("b = " + b);

}
```

Primitive Types in instanceof:

- **Before Java 23:** instanceof only worked with reference types.
- **Since Java 23:** You can directly use instanceof with primitive types for simpler type checks.

Example:

```
int value = 42;
```

```
if (value instanceof Integer) { // Old way (still works)

    System.out.println("value is an Integer");

}
```

```
if (value instanceof int) { // Possible with Java 23

    System.out.println("value is an int");

}
```

```
int value = 10;
```

```
if (value instanceof int i && i > 5) { // Possible with Java 23

    System.out.println("Value is an integer greater than 5");

}
```

Primitive Types in switch expressions:

- **Before Java 23:** switch expressions (introduced in Java 14) only supported byte, short, char and int types.
- **Since Java 23:** We can now use switch expressions with all primitive types (long, float, double, and boolean) for broader applicability.

Examples:

```
double value = 3.14;

switch (value) {

    case int i -> System.out.println("value is an integer");

    case double d -> System.out.println("value is a double"); // Not possible before Java 23

    default -> System.out.println("value is something else");

}
```

```
double d = 3.14;

switch (d) {

    case 3.14 -> System.out.println("Value is PI");

    case 2.71 -> System.out.println("Value is Euler's number");

    default -> System.out.println("Value is not a recognized constant");

}
```

```
float rating = 0.0f;

switch (rating) {

    case 0f -> System.out.println("0 stars");

    case 2.5f -> System.out.println("Average");

    case 5f -> System.out.println("Best");

    default -> System.out.println("Invalid rating");

}
```

Markdown Documentation Comments – JEP 467

Till Java 22, the Java documentation comments were written using a mix of HTML and JavaDoc tags ('@'). Now Java 23 onward, we will also be able to use Markdown to write the JavaDoc comments. This is a final feature in Java 23. This enhancement simplifies writing and reading JavaDoc, making documentation a more user-friendly and readable, clearer and more consistent, especially for developers familiar with Markdown syntax.

How to use Markdown Documentation Comments?

Initiation: Start documentation comments with three forward slashes (`///`) instead of the standard `/** */` comment block.

Paragraphs: Separate paragraphs with blank lines, similar to standard Markdown practice.

Formatting: Utilize Markdown syntax for elements like headings, bold text, italics, and lists [(<https://openjdk.org/jeps/467>)].

Example (Traditional vs. Markdown)

Traditional Approach (Till Java 22)

```
/**
 * This method calculates the factorial of a number.
 * @param n the number to calculate the factorial for
 * @return the factorial of n
 */
public static int factorial(int n) {
    // ...
}
```

Markdown Approach (Since Java 23)

```
/// This method calculates the factorial of a number.
```

```
/// Parameters:
```

```
/// * `n`: the number to calculate the factorial for
```

```
/// Returns:
```

```
/// The factorial of n
```

```
public static int factorial(int n) {
    // ...
}
```

Module Import Declarations (Preview) – JEP 476

Java 23 introduced a new preview feature 'Module Import Declarations (JEP 476)' to simplify the process of importing all public top-level classes and interfaces from a module. Instead of individually importing each class, we can import the entire module at once that reduces verbosity in our code.

Syntax:

```
import module <module_name>;
```

Example:

```
import module java.util; // Imports all public classes and interfaces from java.util
```

```
public class MyClass {
    public void processList() {
        List<String> myList = new ArrayList<>();

        // Use other classes/interfaces from java.util like Map, Set, etc.
    }
}
```

```
}
```

To summarize, instead of:

```
import java.util.List;
```

```
import java.util.Map;
```

```
import java.util.Set;
```

We can write :

```
import module java.util;
```

This reduces the need for multiple package-level imports, streamlining imports when using several classes from a module.

Conflicting Class/Interface Names: Suppose there are more than one class with the same name in a module, then we need to provide explicit import for the specific class in order to avoid any compilation error. For example, observe the below code:

```
import module java.base;
```

```
import module java.sql;
```

```
Date date = new Date(); // Compiler error: "reference to Date is ambiguous"
```

In order to resolve this, we also have to import the specific Date class explicitly:

```
import module java.base;
```

```
import module java.sql;
```

```
import java.util.Date;
```

```
Date date = new Date(); // No Compilation error
```

Implicitly Declared Classes and Instance Main Methods (Third Preview) – JEP 477

The concept of Implicitly Declared Classes and Instance Main Methods as part of Java 23 (Third Preview) offers a simplified approach for writing basic Java programs, particularly beneficial for beginners.

The changes were first published in [JDK 21](#) under the name "Unnamed Classes and Instance Main Methods." In Java 22, this was introduced as a second preview feature and renamed as "Implicitly Declared Classes and Instance Main Methods".

In Java 23, [JEP 477](#) added the automatic import of 'java.io.IO' class in order to eliminate 'System.out' also, which was not yet possible in the second preview in Java 22.

From Java 21:

```
void main() {
```

```
    System.out.println("Hello world!");
```

```
}
```

Since Java 23:

```
void main() {
```

```
    println("Hello world!");
```

```
}
```

Therefore, since Java 23, the above code is a valid and complete Java program.

Flexible Constructor Bodies (Second Preview) – JEP 482

This is the second preview feature. The first preview feature was introduced in Java 22 with the feature name "[Statements Before super\(...\) or this\(...\)](#) [Preview, JEP 447]". It states that we can execute code in constructors before calling `super(...)` or `this(...)`. We can execute any code that does not access the currently constructed instance, i.e., does not access its fields.

In addition, we may initialize the fields of the instance just being constructed. This was made possible in Java 23 by [JDK Enhancement Proposal 482](#).

It allows executing statements and initializing fields before calling the superclass constructor, enabling validation and handling of constructor arguments before object creation. It enhances constructor flexibility by allowing more initialization logic before the call to the superclass constructor. With JEP 482, certain code (such as initializing fields or executing logic) can now precede the superclass constructor call.

The instance can be partially initialized before calling the superclass constructor, but the constructor must eventually call `super()`.

Example#1:

```
public class PositivePoint {  
    public PositivePoint(int x, int y) {  
        if (x < 0 || y < 0) {  
            throw new IllegalArgumentException("Coordinates must be positive");  
        }  
        super(x, y);  
    }  
}
```

Example#2:

```
public class Parent{  
    Parent(int i) {  
        System.out.println("Parent constructor called with: " + i);  
    }  
}
```

```
public class Child extends Parent{
```

```
    int x;
```

```
    Child (int x) {  
        // Flexible logic before super()  
        if (x < 0) {  
            x = 0;  
        }  
        // Now call to super  
        super(x);  
        this.x = x;  
    }  
}
```

Here, field validation (if `(x < 0)`) occurs before the `super()` call, which demonstrates the flexibility in constructor bodies.

Class-File API (Second Preview) – JEP 466

The **Class-File API** (JEP 466) in **Java 23** introduces a comprehensive API for parsing, generating, and transforming Java class files. It is designed to replace third-party libraries and provide a more streamlined and standardized approach that adheres to the JVM specification. Class-File API is a preview feature, meaning it's not ready for production use yet but available for testing and experimentation. Key features of JEP-466 are:

- **Class File Parsing:** The API allows developers to easily read and manipulate .class files to extract metadata like method signatures, fields, and bytecode.
- **Class File Generation:** The API can generate new class files, useful for tools and frameworks that dynamically create classes at runtime.
- **Transformation:** Combines both parsing and generation to facilitate bytecode transformations, enhancing use cases such as instrumentation or proxies.

Example:

```
ClassFile classFile = ClassFile.read("MyClass.class");
```

```
Method method = classFile.methods().get(0);
```

```
System.out.println(method.name()); // Output: methodName
```

This code demonstrates reading a class file and accessing method information, highlighting the ease of parsing class structures.

Stream Gatherers (Second Preview) – JEP 473

Stream Gatherers (JEP 473) is a second preview feature introduced in Java 23. It is a new enhancement to the Stream API that allows for more advanced stream processing by supporting custom intermediate operations. The main goal is to allow developers to perform more flexible transformations on stream elements that would have been difficult or cumbersome with existing operations like map, filter, or reduce. Key features of JEP-473 are:

- **Custom Intermediate Operations:** A gatherer can process elements sequentially until a condition is met, after which the processing behavior may change. This allows for more sophisticated transformations, such as gathering elements into groups.
- **Versatility:** It provides finer control over how elements flow through the stream pipeline, enabling advanced data transformations beyond standard operations.

Example:

```
Stream<Integer> numbers = Stream.of(1, 2, 3, 4, 5, 6);
```

```
Stream<Integer> gathered = numbers.gather((Gatherer<Integer, List<Integer>>) list -> list.size() == 3);
```

```
gathered.forEach(System.out::println); // Output: [1, 2, 3], [4, 5, 6]
```

Here, gather() collects elements into lists of size 3, showcasing how custom grouping can be achieved

Structured Concurrency (Third Preview) – JEP 480

Structured Concurrency (JEP 480), introduced as a Third Preview in Java 23, is aimed at simplifying multi-threaded programming. It treats groups of related tasks running in different threads as a single unit of work. Hence, it improves the way developers manage concurrent tasks and streamline error handling. Key features of JEP- 480 include:

- **Task Grouping:** By treating multiple threads as one unit, errors in any thread are propagated and handled together.
- **Enhanced Error Handling:** If any task in the group fails, the rest of the tasks are either canceled or managed efficiently, reducing the complexity of error recovery.
- **Improved Debugging:** Since all tasks in a group are structured, their lifecycle is more predictable, simplifying the debugging process.

Example:

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
```

```
    Future<String> result1 = scope.fork(() -> task1());
```

```
    Future<String> result2 = scope.fork(() -> task2());
```

```
    scope.join(); // Wait for both tasks
```

```
    scope.throwIfFailed(); // Propagate error if any
```

```
    String combinedResult = result1.resultNow() + result2.resultNow();
```

```
    System.out.println(combinedResult);
```

```
}
```

In this example, both tasks are executed concurrently, and any failure in one task will propagate the error and handle it for the group

Scoped Values (Third Preview) – JEP 481

Scoped Values (JEP 481), introduced as a Third Preview in Java 23, offers a way for methods to share immutable data with their callees within the same thread, as well as across child threads. The key purpose is to avoid passing data manually between methods, especially in complex call chains, thus simplifying the flow of data in concurrent applications. Key features of JEP-481 include:

- **Immutable Data Sharing:** Scoped values are immutable, ensuring thread safety when accessed by different threads.
- **Avoids Parameter Passing:** Methods do not need to explicitly pass data through parameters, improving code readability.
- **Data Sharing Across Child Threads:** Scoped values are propagated to child threads, facilitating better management of concurrent tasks.

Example:

```
public class ScopedValueExample {
```

```
    public static final ScopedValue<String> USER_ID = ScopedValue.newInstance();
```

```

public static void main(String[] args) {

    ScopedValue.where(USER_ID, "12345").run() -> {

        System.out.println("User ID: " + USER_ID.get());

        Thread(() -> System.out.println("Child Thread User ID: " + USER_ID.get())).start();

    });

}
}

```

In this example, `USER_ID` is shared between the main thread and a child thread, simplifying data access without manual passing of parameters

Vector API (Eighth Incubator) – JEP 469

The Vector API (Eighth Incubator), introduced in JEP 469, allows Java developers to write efficient vector computations that can be automatically optimized to take advantage of modern CPU vector instructions. This feature aims to improve performance in operations involving large datasets, particularly in scientific or financial applications. Key features of JEP-469 include:

- **Platform-Independent:** The API ensures that the code can be written once and deployed on different hardware, with the JVM optimizing it for the underlying CPU architecture.
- **Better Performance:** By leveraging vector instructions, the API can perform the same operation on multiple data points simultaneously, leading to significant performance gains.
- **Eighth Incubator:** This iteration continues refining the API to ensure compatibility with more hardware and CPUs.

Example:

```

import jdk.incubator.vector.*;

public class VectorExample {

    public static void main(String[] args) {

        var species = FloatVector.SPECIES_256;

        var vectorA = FloatVector.fromArray(species, new float[]{1.0f, 2.0f, 3.0f, 4.0f}, 0);

        var vectorB = FloatVector.fromArray(species, new float[]{5.0f, 6.0f, 7.0f, 8.0f}, 0);

        var result = vectorA.add(vectorB);

        result.intoArray(new float[4], 0);

        System.out.println(result);

    }

}

```

This code performs vector addition on two float arrays using the Vector API, optimizing the computation at runtime

ZGC: Generational mode by default – JEP 474

JEP 474, titled **ZGC: Generational Mode by Default**, introduces a significant enhancement to the Z Garbage Collector (ZGC) in Java 23. This feature makes generational mode the default operation mode for ZGC, improving its efficiency in garbage collection by organizing objects into two generations: young and old. The key points of JEP-474 are:

- **Generational Mode:** In this mode, objects are separated into two generations. New objects are allocated in the young generation, which is collected more frequently. The old generation holds longer-lived objects.
- **Performance Improvement:** This change leverages the weak generational hypothesis, which posits that most objects die young. By focusing on the young generation, ZGC can minimize pause times and improve overall performance during garbage collection.
- **Default Behavior:** Previously, ZGC managed the heap as a single generation. JEP 474 sets the generational option to true by default, enhancing performance for applications with varying object lifetimes.

Example:

Here is a simple command to enable ZGC in generational mode when running a Java application:

```
java -XX:+UseZGC -XX:ZGenerational=true -jar myApplication.jar
```

This command ensures that ZGC operates in generational mode, optimizing garbage collection performance for the application.